Investigation and Analysis of Computing Data for Data Management

Nishnat Acharya
S1-s1-E39C

Bsc Hons Ethical Hacking & Cyber Security, Softwarica College of IT &

E-commerce

ST4015CMD Foundations of Computer Science

Submitted to Rupak Rajbansi

5th Mar 2026

Table of Contents

Task 1: Enhancing Secure Data Exchange with Encoding Formats and Protocol Integration..... 4
1.1 What is Encoding and Why Does it Matter?..... 5
1.2 Strengths and Weaknesses of Key Encoding Formats..... 5
- Base64.....5
- Hexadecimal (Hex) Encoding..... 5
URL Encoding (Percent Encoding).....6
1.3 Data Flow: Base64 in TLS-Based Email Transmission..... 7
1.4 Encoding in HTTP Transmission and Injection Attack Prevention.....8
1.5 Interoperability with REST APIs and OAuth..... 8
1.6 Risks of Encoding-Based Obfuscation by Attackers.....9
1.7 Proposed Enhanced Encoding Strategies..... 9
Task 2: Classroom Seating Arrangement Problem — P vs NP and Algorithmic Approaches.....10
2.1 What Are P and NP Problems?..... 11
2.2 Applying P vs NP to the Seating Problem.....11
Why Checking a Seating Plan is Easy..... 11
Why Finding a Correct Seating Plan is Hard..... 12
2.3 Brute Force Approach.....12
2.4 Heuristic (Smart) Approach..... 13
Heuristic 1: Most Constrained First.....13
Heuristic 2: Separate Same-City Students Early..... 14
Task 3: College Club Membership Management — Database Normalisation.....15
3.1 The Original Table and Its Problems.....15
Problem 1: Data Redundancy.....17
Problem 2: Duplicate Data and Update Anomaly..... 17
Problem 3: Insert and Deletion Anomalies.....17
3.2 Normalisation — Making the Data Better..... 18
Step 1: First Normal Form (1NF)..... 18
Step 2: Second Normal Form (2NF)..... 18
Step 3: Third Normal Form (3NF)..... 21
3.3 Entity Relationship (ER) Diagram..... 21
3.4 Basic SQL Operations..... 22
Creating the Tables.....22
Task 4: Insert and Select Queries.....23
Task 5: SQL JOIN Query..... 23
3.5 Reflection: Benefits of Normalisation and the Need for JOINs.....24
How Normalisation Reduces Data Repetition.....24
Why JOINs Are Necessary After Normalisation.....24

List of Figures

Figure 1: Comparison of Common Encoding Formats.....7
Figure 2: Data Flow of Base64 Encoding Through TLS-Based Email Transmission.....8
Figure 3: Comparison of P and NP Problem Classes..... 12
Figure 4: Growth of Brute Force Complexity with Number of Students..... 13
Figure 5: Greedy Heuristic Algorithm for Classroom Seating..... 15
Figure 6: Original Single-Table Design for Club Membership.....17
Figure 7: Summary of Data Problems in the Original Table..... 18
Figure 8: Normalised Student Table (2NF)..... 19
Figure 9: Normalised Club Table (2NF).....20
Figure 10: Normalised Membership Table (2NF)..... 21
Figure 11: Entity Relationship (ER) Diagram for Club Membership System.....22

Task 1: Enhancing Secure Data Exchange with Encoding Formats and Protocol Integration

Data encoding is one of the foundational ideas that keeps the modern internet running smoothly. Whenever you log into a website, send an email, or upload a file, the data you send goes through a process of encoding before it is transmitted. This section explores the most important encoding formats Base64, Hex, and URL encoding and examines how they work together with protocols like HTTPS, TLS, and SMTP to enable secure and efficient communication.

1.1 What is Encoding and Why Does it Matter?

Encoding is the process of converting data from one format into another so that it can be safely transmitted or stored. It is important to understand that encoding is not the same as encryption. Encryption hides the meaning of data, whereas encoding simply changes the representation. For example, the letter 'A' can be encoded in ASCII as the number 65, or in Base64 it might appear as part of a longer string. Anyone who knows the encoding scheme can reverse the process and get the original data back.

Encoding matters because different systems and protocols were built with different assumptions about what kind of data they can handle. Email systems, for instance, were designed to carry plain text. When you want to attach a photo or a document, the binary data must be encoded into a text-friendly format before the email protocol can carry it. This is where Base64 comes in.

1.2 Strengths and Weaknesses of Key Encoding Formats

- Base64

Base64 works by taking three bytes of binary data and turning them into four printable ASCII characters. The name comes from the fact that it uses 64 characters: A-Z, a-z, 0-9, and the symbols + and /. This makes it safe to use in environments that only support text, like email bodies or JSON payloads.

Strengths: Base64 is universally supported across programming languages and platforms. It is simple to implement and reliable for carrying binary data such as images, certificates, or keys through text-based protocols. It is used in MIME email attachments, Basic HTTP Authentication headers, and embedding images in HTML pages (data URIs).

Weaknesses: Base64 increases the size of data by approximately 33%. A 100 KB image becomes roughly 133 KB after encoding. It also provides no confidentiality — any attacker who intercepts the data and knows it is Base64-encoded can decode it immediately with freely available tools.

- Hexadecimal (Hex) Encoding

Hex encoding converts each byte of data into two hexadecimal characters (0-9 and A-F). For example, the letter 'A' (ASCII 65, binary 01000001) becomes '41' in hexadecimal. This makes it extremely readable for debugging and for representing cryptographic values such as hash outputs, certificate fingerprints, and key values.

Strengths: Hex is very easy to inspect visually, which is why developers use it to examine raw network packets, cryptographic keys, and checksums. It is precise every byte maps to exactly two characters.

Weaknesses: Hex encoding doubles the size of data. A 1 MB file becomes 2 MB when encoded as hex. Like Base64, hex provides zero security by itself.

URL Encoding (Percent Encoding)

URLs can only contain a limited set of characters. Characters like spaces, ampersands (&), equals signs (=), and non-ASCII characters would break the structure of a URL if included directly. URL encoding (also called percent encoding) replaces these characters with a percent sign followed by their two-digit hexadecimal code. A space becomes %20, a forward slash becomes %2F, and so on.

Strengths: URL encoding ensures that query parameters and paths are transmitted correctly across web browsers and servers, preventing characters from being misinterpreted as control characters in the URL.

Weaknesses: URL encoding can be exploited. Attackers sometimes use double encoding (encoding the percent sign itself) to bypass input validation filters. For example, %252F is a double-encoded forward slash. If a server decodes input twice, the second decode reveals the slash, potentially enabling path traversal attacks.

Format	Typical Use	Size Overhead	Security Level
Base64	Email attachments, HTTP Auth, JWTs	+33%	None (reversible)
Hexadecimal	Crypto hashes, debugging, checksums	+100%	None (reversible)
URL Encoding	Query strings, web form data	Variable	None (reversible)

Figure 1: Comparison of Common Encoding Formats

1.3 Data Flow: Base64 in TLS-Based Email Transmission

To understand how encoding fits into real-world secure communication, consider the journey of an email with an attachment. The diagram below traces the data flow step by step:

Step	Layer/Actor	What Happens
Step 1	Client (Browser)	Plaintext data: user credentials or file bytes
Step 2	Encoding Layer	Data encoded using Base64 (binary → ASCII safe string)
Step 3	TLS Handshake	Server & client exchange certificates; symmetric key established
Step 4	Encryption	Base64-encoded payload encrypted using AES (symmetric key)

Step 5	HTTPS Transmission	Encrypted data travels over the network securely
Step 6	Server Side	Data decrypted → Base64 decoded → original data recovered

Figure 2: Data Flow of Base64 Encoding Through TLS-Based Email Transmission

At step one, the user composes an email and attaches a file. The email client reads the file as raw binary bytes. At step two, the MIME (Multipurpose Internet Mail Extensions) standard requires the attachment to be encoded using Base64 before it can be placed inside the email body, because SMTP was designed for plain text. The Base64-encoded string is then placed in the email's MIME section with a header like 'Content-Transfer-Encoding: base64'. At step three, before the email is sent, the email client and server perform a TLS handshake they verify each other's identity using certificates and agree on a symmetric encryption key. At step four, the entire email (including the Base64-encoded attachment) is encrypted using AES or another symmetric cipher. Finally, the encrypted data travels over the network, and the receiving server decrypts it, then decodes the Base64 to recover the original file.

1.4 Encoding in HTTP Transmission and Injection Attack Prevention

When a web browser submits a form, the data in the form fields is URL-encoded before being sent in an HTTP POST request. For example, a search for 'laptop & accessories' becomes 'laptop+%26+accessories' in the request body. This ensures that special characters do not corrupt the request structure.

Encoding also plays a role in preventing injection attacks. In SQL injection, an attacker tries to insert malicious SQL code into a form field. Proper output encoding (escaping special characters before they reach the database) prevents this. Similarly, in cross-site scripting (XSS), HTML encoding converts characters like < and > into < and > so they are displayed as text rather than executed as HTML tags.

However, encoding alone is not a complete defence. Developers must also use parameterised queries for SQL and a Content Security Policy (CSP) for web pages. Encoding is one layer in a defence-in-depth strategy.

1.5 Interoperability with REST APIs and OAuth

Modern web applications rely heavily on REST APIs to communicate between services. Encoding formats are used throughout these interactions. JSON Web Tokens (JWTs), which are used in OAuth 2.0 authentication, consist of three Base64-URL encoded sections: header, payload, and signature. Base64-URL encoding is a slight variant of standard Base64 that uses - and _ instead of + and / to make it safe for use in URLs without further percent-encoding.

When a user logs in with OAuth, the authorisation server generates a JWT, encodes it in Base64-URL, and returns it to the client. The client includes this token in the Authorization HTTP header with every subsequent API request. The server decodes it and verifies the signature to authenticate the request. This shows how encoding directly enables stateless, scalable authentication in modern web systems.

1.6 Risks of Encoding-Based Obfuscation by Attackers

While encoding is legitimate and necessary, attackers also use it to hide malicious payloads and evade detection systems. A common technique is to encode malware or attack scripts in Base64 and embed them in web pages or email bodies. When the victim's browser or email client decodes the Base64, it executes the malicious code. Security tools that only scan for known attack signatures in plain text will miss the encoded version.

Attackers also use multiple layers of encoding Base64 inside URL encoding inside HTML entities to make detection even harder. This is known as encoding obfuscation. Intrusion detection systems (IDS) must decode all layers of encoding before analysing the content for threats.

1.7 Proposed Enhanced Encoding Strategies

To improve the security of encoding-based data exchange, the following strategies are recommended:

- Content-aware encoding validation: Web application firewalls should decode and inspect all incoming data across multiple encoding layers before applying security rules.
- Strict MIME type enforcement: Servers should reject email attachments or API payloads that do not match their declared content type, reducing the risk of disguised malicious files.
- Token binding for JWTs: OAuth tokens should be cryptographically bound to the TLS session using token binding (RFC 8471), preventing token theft and replay attacks.
- Compressed encoding: Where bandwidth is a concern, Base64-encoded data should be compressed (using gzip) before encoding to partially offset the 33% overhead.

Task 2: Classroom Seating Arrangement Problem — P vs NP and Algorithmic Approaches

The classroom seating problem described in this task is a wonderful example of a constraint satisfaction problem. It looks simple on the surface just arrange a handful of students in seats but as the number of students grows, it becomes surprisingly hard to solve. This section explores the theoretical computer science concepts of P and NP, applies them to the seating scenario, and then examines both a brute-force approach and a smarter heuristic method.

2.1 What Are P and NP Problems?

In computer science, we classify problems based on how difficult they are for a computer to solve. The two most important classes are P and NP.

P (Polynomial time): A problem is in class P if a computer can find the solution in a reasonable amount of time more precisely, if the time taken grows as a polynomial function of the input size. For example, sorting a list of names is a P problem. If you have n names, it takes roughly n log n steps, which grows manageably even for large n.

NP (Nondeterministic Polynomial time): A problem is in class NP if, given a proposed solution, a computer can check whether it is correct in polynomial time. Finding the solution might be very hard, but verifying it is easy. The famous unsolved question in mathematics and computer science is whether P = NP that is, whether every problem that is easy to verify is also easy to solve.

Feature	P Problems	NP Problems
Finding solution	Fast (polynomial time)	Potentially very slow
Checking solution	Fast	Fast (this is the definition of NP)
Examples	Sorting, shortest path, multiplication	Seating arrangements, Sudoku, travelling salesman

Tractable for large n?	Yes	Generally No

Figure 3: Comparison of P and NP Problem Classes

2.2 Applying P vs NP to the Seating Problem

Why Checking a Seating Plan is Easy

Suppose the teacher is given a completed seating arrangement by a student assistant. Checking whether the arrangement is valid is straightforward: go through each pair of adjacent seats and check two things — are these two students friends, and are they from the same city? If neither condition is violated for any adjacent pair, the arrangement is valid. With n students in a row, there are n-1 adjacent pairs to check. This check can be done in linear time, O(n), which is polynomial. Checking is therefore a P-type operation.

Why Finding a Correct Seating Plan is Hard

Finding a valid arrangement from scratch is a completely different matter. The teacher must try to find one arrangement, out of all possible arrangements, that satisfies the constraints. With just 5 students, there are 5! = 120 possible arrangements. With 10 students, there are 10! = 3,628,800 arrangements. With 20 students, there are 20! = 2,432,902,008,176,640,000 arrangements a number larger than the estimated number of seconds since the Big Bang. No computer can realistically check all of these for large n.

The seating problem is closely related to a well-known NP problem called Graph Colouring or the Hamiltonian Path problem, depending on how the constraints are modelled. If we model students as nodes in a graph, with edges between friends or same-city students, the problem of finding a valid linear arrangement becomes equivalent to finding a Hamiltonian path in a constrained graph which is NP-complete (Garey & Johnson, 1979).

2.3 Brute Force Approach

The most straightforward way to solve the seating problem is by brute force: generate every possible arrangement of students and check each one against the rules. This is called exhaustive search or brute-force enumeration.

Number of Students (n)	Possible Arrangements (n!)	Time to Check All (at 1M/sec)
5	120	< 1 second
10	3,628,800	~3.6 seconds
15	1,307,674,368,000	~15 days
20	2.4×10^{18}	~77,000 years
25	1.5×10^{25}	Longer than age of universe

Figure 4: Growth of Brute Force Complexity with Number of Students

As the table shows, brute force works acceptably for very small classes perhaps 5 to 8 students but becomes completely impractical as soon as the class size grows beyond about 12 or 13 students. The time complexity is O(n!) which is called factorial time and grows far faster than even exponential time (2^n). This illustrates why NP problems are so challenging in practice.

2.4 Heuristic (Smart) Approach

Since the brute force method becomes impractical for large classes, the teacher needs a smarter strategy. A heuristic is a practical problem-solving technique that does not guarantee the optimal or correct solution but finds a good enough solution much more quickly. Two sensible heuristics for this problem are described below.

Heuristic 1: Most Constrained First

Place the student who has the most constraints first. A student with many friends in the class is the hardest to seat, because there are many people they cannot sit next to. By placing the most constrained student first, we avoid getting stuck later. This is similar to the 'fail first' principle used in constraint programming.

Heuristic 2: Separate Same-City Students Early

If several students are from the same city, spread them out early in the process. Place one of them at seat 1, then skip at least one seat before placing the next same-city student. This prevents same-city students from clustering together.

Step	Action
Step 1	List all students with their friend-count and city
Step 2	Pick student with most friends; place them in seat 1
Step 3	Place next student: not a friend, not from same city
Step 4	Continue greedy placement from left to right
Step 5	If stuck, backtrack one seat and try next candidate
Step 6	Output the arrangement if all seats filled without conflict

Figure 5: Greedy Heuristic Algorithm for Classroom Seating

The greedy heuristic runs in roughly $O(n^2)$ time in the worst case (choosing each seat requires scanning remaining students), which is vastly better than $O(n!)$. For 100 students, $O(n^2)$ is 10,000 operations versus $O(n!)$ which is a number so large it has no practical meaning.

The limitation of heuristics is that they are not guaranteed to find a valid arrangement even when one exists. If the greedy method gets stuck (all remaining students violate the constraints for the current seat), it may need to backtrack or declare failure. However, for most real classroom scenarios, a greedy heuristic will find an acceptable arrangement quickly.

Task 3: College Club Membership Management — Database Normalisation

Good data management is at the heart of reliable information systems. When data is stored without proper organisation, problems like inconsistency, redundancy, and anomalies emerge. This section analyses the problems in the given club membership table and shows how database normalisation resolves these issues step by step.

3.1 The Original Table and Its Problems

The college currently stores all club-related information in a single flat table. The original data is shown below:

StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	1/10/2024
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	1/12/2024
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	1/15/2024
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	1/20/2024
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	1/18/2024
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	1/22/2024

2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	1/25/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	1/27/2024
3	Nisha	nisha@email.com	Coding Club	Lab1	Mr. Anil	1/28/2024
7	Aman	aman@email.com	Coding Club	Lab1	Mr. Anil	1/30/2024

Figure 6: Original Single-Table Design for Club Membership

Problem 1: Data Redundancy

Asha (StudentID 1) appears in two rows because she has joined two clubs. Her name, email, and StudentID are repeated. Similarly, the information about Music Club (room R101, mentor Mr. Raman) appears three times. This is redundant data the same information is stored multiple times unnecessarily. Redundancy wastes storage space and makes updates difficult.

Problem 2: Duplicate Data and Update Anomaly

If Mr. Raman is replaced as the Music Club mentor, the update must be made in all three rows where Music Club appears. If even one row is missed, the database becomes inconsistent — some rows say Mr. Raman while others say the new mentor. This is called an update anomaly.

Problem 3: Insert and Deletion Anomalies

An insert anomaly occurs when it is impossible to add new information without also adding other unrelated information. For example, if a new club (Photography Club) is created but no students have joined yet, it cannot be added to this table because every row requires a student. A deletion anomaly occurs when deleting one piece of information accidentally removes other unrelated information. If Rohan (StudentID 4)

leaves the Drama Club, his row is deleted, and with it, all information about the Drama Club (room R303, mentor Mr. Kiran) is lost from the database.

Anomaly Type	Description	Example from the Table
Redundant Data	Same data stored multiple times	Music Club info repeated 3 times
Update Anomaly	Updating data in one row but not others	Changing Music Club mentor requires 3 updates
Insert Anomaly	Cannot add data without unrelated data	Cannot add Photography Club without a student
Deletion Anomaly	Deleting a row removes unrelated data	Deleting Rohan removes Drama Club information

Figure 7: Summary of Data Problems in the Original Table

3.2 Normalisation — Making the Data Better

Normalisation is the process of organising a database to reduce redundancy and improve data integrity. It follows a series of rules called Normal Forms. We will apply the first three normal forms (1NF, 2NF, and 3NF) to the club membership data.

Step 1: First Normal Form (1NF)

A table is in First Normal Form if every cell contains a single (atomic) value, every column has a unique name, and there are no repeating groups. The original table already satisfies the atomicity requirement (each cell holds one value), but there is no clear primary key. The StudentID alone is not unique across rows (Asha appears twice with StudentID 1). The combination of StudentID and ClubName is unique and serves as a composite primary key. After establishing this key, the table is in 1NF.

Step 2: Second Normal Form (2NF)

A table is in Second Normal Form if it is in 1NF and every non-key column is fully dependent on the entire primary key (not just part of it). In our 1NF table, the primary key is (StudentID, ClubName). However, StudentName and Email depend only on StudentID, and ClubRoom and ClubMentor depend only on ClubName. These are partial dependencies, which violate 2NF. To fix this, we split the table into three separate tables:

Student Table:

StudentID (PK)	StudentName	Email
1	Asha	asha@email.com
2	Bikash	bikash@email.com
3	Nisha	nisha@email.com
4	Rohan	rohan@email.com
5	Suman	suman@email.com
6	Pooja	pooja@email.com
7	Aman	aman@email.com

Figure 8: Normalised Student Table (2NF)

Club Table:

ClubID (PK)	ClubName	ClubRoom	ClubMentor
C01	Music Club	R101	Mr. Raman
C02	Sports Club	R202	Ms. Sita
C03	Drama Club	R303	Mr. Kiran
C04	Coding Club	Lab1	Mr. Anil

Figure 9: Normalised Club Table (2NF)

Membership Table:

MemberID (PK)	StudentID (FK)	ClubID (FK)	JoinDate
M01	1	C01	1/10/2024
M02	2	C02	1/12/2024
M03	1	C02	1/15/2024
M04	3	C01	1/20/2024
M05	4	C03	1/18/2024

M06	5	C01	1/22/2024
M07	2	C03	1/25/2024
M08	6	C02	1/27/2024
M09	3	C04	1/28/2024
M10	7	C04	1/30/2024

Figure 10: Normalised Membership Table (2NF)

Step 3: Third Normal Form (3NF)

A table is in Third Normal Form if it is in 2NF and there are no transitive dependencies — that is, no non-key column depends on another non-key column. In the Club table, we need to verify that ClubRoom and ClubMentor depend directly on ClubID (the primary key) and not on each other. If ClubMentor were determined by ClubRoom (i.e., each room always has the same mentor), that would be a transitive dependency. In our case, ClubMentor is assigned to the club (not the room), so there is no transitive dependency. All three tables are now in 3NF.

In the Student table, StudentName and Email depend directly on StudentID with no transitive dependencies. In the Membership table, JoinDate depends on the combination of StudentID and ClubID (the composite key), which is appropriate.

3.3 Entity Relationship (ER) Diagram

The ER diagram below represents the relationships between the three entities in our normalised database. The key relationships are: one student can join many clubs (a student appears multiple times in the Membership table), and one club can have many students.

Entity	Attributes	Cardinality	Relationship
Student	StudentID (PK), StudentName, Email	-	StudentID → Membership
Club	ClubID (PK), ClubName, ClubRoom, ClubMentor	-	ClubID → Membership
Membership	MembershipID (PK), StudentID (FK), ClubID (FK), JoinDate	Many-to-Many	Links Student and Club

Figure 11: Entity Relationship (ER) Diagram for Club Membership System

In the ER model, Student and Club are the two main entities, and Membership is the associative entity (also called a junction or bridge table) that resolves the many-to-many relationship between them. StudentID in the Membership table is a foreign key referencing the Student table, and ClubID is a foreign key referencing the Club table.

3.4 Basic SQL Operations

Creating the Tables

```
-- Create Student Table

CREATE TABLE Student (

    StudentID INT PRIMARY KEY,

    StudentName VARCHAR(100),

    Email VARCHAR(100)

);

CREATE TABLE Club (

    ClubID VARCHAR(10) PRIMARY KEY,

    ClubName VARCHAR(100),
```

```
    ClubRoom VARCHAR(20),
    ClubMentor VARCHAR(100)
);
CREATE TABLE Membership (
    MemberID VARCHAR(10) PRIMARY KEY,
    StudentID INT REFERENCES Student(StudentID),
    ClubID VARCHAR(10) REFERENCES Club(ClubID),
    JoinDate DATE
);
```

Task 4: Insert and Select Queries

```
-- Insert a new student
INSERT INTO Student (StudentID, StudentName, Email)
VALUES (8, 'Priya', 'priya@email.com');
-- Insert a new club
INSERT INTO Club (ClubID, ClubName, ClubRoom, ClubMentor)
VALUES ('C05', 'Photography Club', 'Lab2', 'Ms. Devi');
-- Display all students
SELECT * FROM Student;
-- Display all clubs
SELECT * FROM Club;
```

Task 5: SQL JOIN Query

After normalisation, data is spread across multiple tables. To get a combined view — for example, to see which student joined which club and when — we need to JOIN the

tables together. The JOIN operation links rows from different tables based on shared key values.

```
-- Display Student Name, Club Name, and Join Date

SELECT s.StudentName, c.ClubName, m.JoinDate

FROM Membership m

JOIN Student s ON m.StudentID = s.StudentID

JOIN Club c ON m.ClubID = c.ClubID;
```

3.5 Reflection: Benefits of Normalisation and the Need for JOINs

How Normalisation Reduces Data Repetition

Before normalisation, the Music Club's room and mentor information was stored in three separate rows. After normalisation, this information is stored once in the Club table. If the mentor changes, only one row needs to be updated. This eliminates update anomalies and ensures consistency across the entire database.

Normalisation also removes insertion and deletion anomalies. A new club can be added to the Club table without needing any student membership data. A student can leave a club (deleting a row from Membership) without losing the club's details.

Why JOINs Are Necessary After Normalisation

Normalisation spreads data across multiple tables to eliminate redundancy. This is efficient for storage and updates, but it means that retrieving a combined view of the data (such as 'which students are in which clubs') requires re-assembling information from multiple tables. The SQL JOIN operation performs this reassembly. The JOIN works by matching rows from the Student and Club tables to rows in the Membership table using the shared foreign key values (StudentID and ClubID). JOINs are therefore not a complication introduced by normalisation they are the mechanism that allows normalised data to be queried flexibly.

References:

Task 1 – Encoding Formats & Secure Data Exchange

[1] Josefsson, S. (2006). The Base16, Base32, and Base64 Data Encodings. RFC 4648. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc4648 [Accessed 1 March 2026].

[2] Forouzan, B.A. (2012). Data Communications and Networking. 5th edn. New York: McGraw-Hill.

[3] Klensin, J. (2008). Simple Mail Transfer Protocol. RFC 5321. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc5321 [Accessed 1 March 2026].

[4] OWASP (2021). OWASP Top 10: Injection. Open Web Application Security Project. Available at: https://owasp.org/Top10/A03_2021-Injection/ [Accessed 2 March 2026].

[5] Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446. Internet Engineering Task Force. Available at: https://www.rfc-editor.org/rfc/rfc8446 [Accessed 1 March 2026].

Task 2 – P vs NP & Seating Arrangement Problem

[6] Sipser, M. (2012). Introduction to the Theory of Computation. 3rd edn. Boston: Cengage Learning.

[7] Garey, M.R. and Johnson, D.S. (1979). Computers and Intractability: A Guide to the Theory of NP-Completeness. San Francisco: W.H. Freeman.

Task 3 – Database Normalisation, ER Modelling & SQL

[8] Codd, E.F. (1970). 'A Relational Model of Data for Large Shared Data Banks', Communications of the ACM, 13(6), pp.377–387. Available at: https://doi.org/10.1145/362384.362685

[9] Date, C.J. (2003). An Introduction to Database Systems. 8th edn. Boston: Addison-Wesley.

[10] Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019). Database System Concepts. 7th edn. New York: McGraw-Hill Education.

[11] Bhusal, G. (2025). Foundation of Computer Science: Module Lecture Notes. Softwarica College of IT & E-Commerce, Kathmandu.